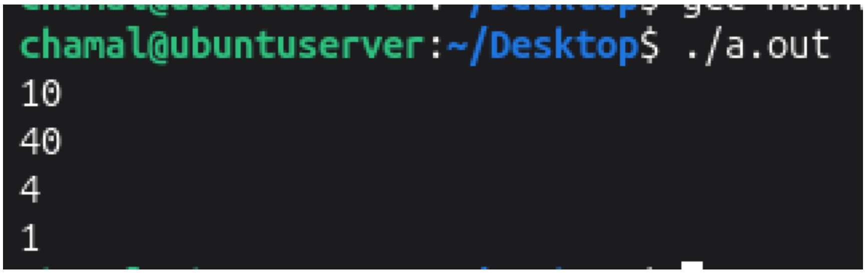
Registration No. : 2025/CS/079

Index No. : 25000799

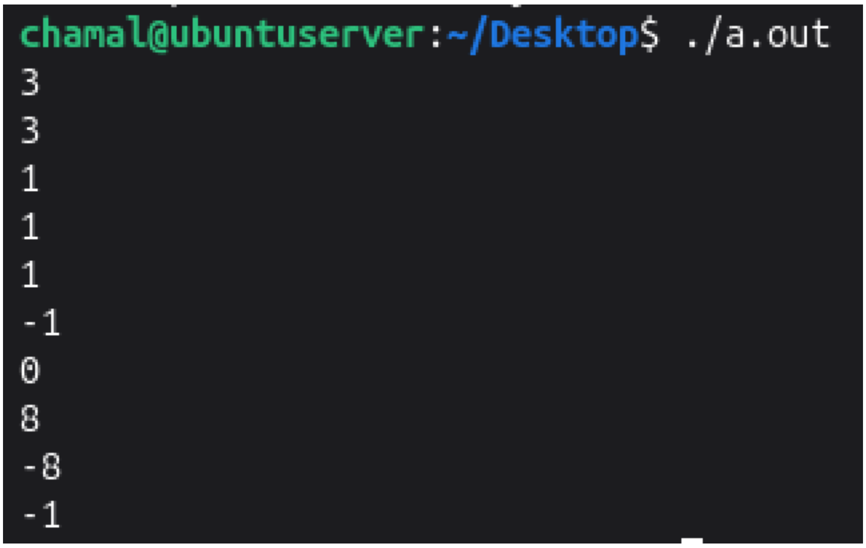
(1)

10
40
4
1



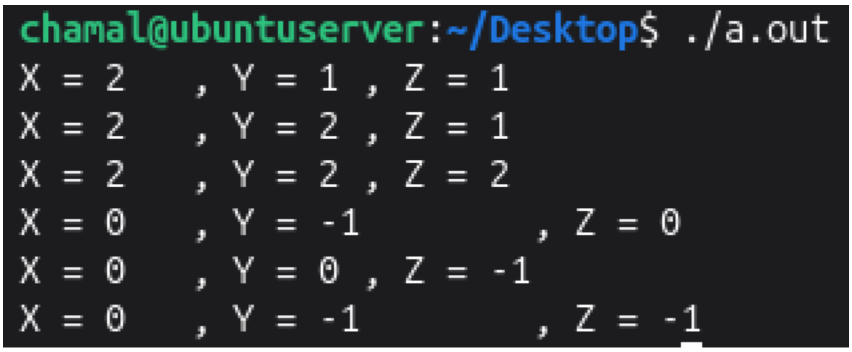
(2)

03 → 011
02 → 010
01 → 001
-01 → 111
Output:
3
3
1
1
1
-1
0
8
-8
-1



(3)

X = 2 , Y = 1 , Z = 1
X = 2 , Y = 2 , Z = 1
X = 2 , Y = 2 , Z = 2
X = 0 , Y = -1 , Z = 0
X = 0 , Y = 0 , Z = -1
X = 0 , Y = -1 , Z = -1



(4)

a. FALSE → It starts printing from where the cursor is
b. TRUE → All variables must be defined with a data type before using them
c. FALSE → All variables must have a data type
d. FALSE → % operator can be used with char as well.
e. FALSE → * and / has a higher precedence than + and -
f. FALSE → Some operators work right to left (Ex. Unary operators)
g. FALSE → You can use a single printf function associated with \n character
h. TRUE → Integer division is truncating the precision points after a division
i. FALSE → Variable names are case sensitive in C
j. TRUE → The function printf is declared inside the stdio.h header file.

(5)

a.

```c
#include <stdio.h>

int main() {
    int a = 40;
    int b = 35;
    int c = -12;

    if (a > b && a > c) {
        a = (a > b && a > c);
    } else if (b > a && b > c) {
        b = (b > a && b > c);
    } else if (c > b && c > a) {
        c = (c > b && c > a);
    }
    printf("a = %d\tb = %d\tc = %d\n", a, b, c);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
a = 1    b = 35   c = -12
```

b.

```c
#include <stdio.h>

int main() {
    int a = 40;
    int b = 35;
    int c = -12;

    if (a < b && a < c) {
        a = (a < b && a < c);
    } else if (b < a && b < c) {
        b = (b < a && b < c);
    } else if (c < b && c < a) {
        c = (c < b && c < a);
    }
    printf("a = %d\tb = %d\tc = %d\n", a, b, c);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
a = 40   b = 35   c = 1
```

(6)

a.

```c
#include <stdio.h>

int main() {
    float cel = 100.0f;
    float fah;

    fah = ((9.0 / 5.0) * cel) + 32.0;

    printf("%.2fC = %.2fF\n", cel, fah);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
100.00C = 212.00F
```

b.

```c
#include <stdio.h>

#define HEART_RATE_CONSTANT 220

int main () {
    int age = 21;
    int max_heart_rate = HEART_RATE_CONSTANT - age;
    float maximum_heart_rate = max_heart_rate * (85.0 / 100.0);
    float minimum_heart_rate = max_heart_rate * (50.0 / 100.0);

    printf("Maximum Heart Rate : %.2f\nMinimum Heart Rate : %.2f\n", maximum_heart_rate, minimum_heart_rate);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
Maximum Heart Rate : 169.15
Minimum Heart Rate : 99.50
```

c.

```c
#include <stdio.h>

int main() {
    int a = 5 > 3;
    int b = 'a' == 'b';
    int c = 3 + 1 == 5;
    int d = 10 >= 10;
    int e = 5 + 5 == 10;
    int f = 12 && 0;
    int g = 0 || 3;
    int h = 12 + 43 < 54;
    int i = !123;
    int j = 'n' > 'a';

    printf("A = %d\tB = %d\tC = %d\tD = %d\tE = %d\tF = %d\tG = %d\tH = %d\tI = %d\tJ = %d\n", a, b, c, d, e, f, g, h, i, j);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
A = 1   B = 0   C = 0   D = 1   E = 1   F = 0   G = 1   H = 0   I = 0   J = 1
```

d.

```c
#include <stdio.h>
#include <limits.h>

int main() {
    int max_int = INT_MAX;
    int min_int = INT_MIN;
    short max_short = SHRT_MAX;
    short min_short = SHRT_MIN;
    long max_long = LONG_MAX;
    long min_long = LONG_MIN;

    // complements
    printf("Complements\n\tIntegers\n\t\tMaximum --> ~%d = %d\n\t\tMinimum --> ~%d = %d\n", max_int, ~max_int, min_int, ~min_int);
    printf("\tShorts\n\t\tMaximum --> ~%d = %d\n\t\tMinimum --> ~%d = %d\n", max_short, ~max_short, min_short, ~min_short);
    printf("\tLongs\n\t\tMaximum --> ~%ld = %ld\n\t\tMinimum --> ~%ld = %ld\n", max_long, ~max_long, min_long, ~min_long);
    // left shift
    printf("Left Shift\n\tIntegers\n\t\tMaximum --> %d<<2 = %d\n\t\tMinimum --> %d<<2 = %d\n", max_int, max_int<<2, min_int, min_int<<2);
    printf("\tShorts\n\t\tMaximum --> %d<<2 = %d\n\t\tMinimum --> %d<<2 = %d\n", max_short, max_short<<2, min_short, min_short<<2);
    printf("\tLongs\n\t\tMaximum --> %ld<<2 = %ld\n\t\tMinimum --> %ld<<2 = %ld\n", max_long, max_long<<2, min_long, min_long<<2);
    // right shift
    printf("Right Shift\n\tIntegers\n\t\tMaximum --> %d>>2 = %d\n\t\tMinimum --> %d>>2 = %d\n", max_int, max_int>>2, min_int, min_int>>2);
    printf("\tShorts\n\t\tMaximum --> %d>>2 = %d\n\t\tMinimum --> %d>>2 = %d\n", max_short, max_short>>2, min_short, min_short>>2);
    printf("\tLongs\n\t\tMaximum --> %ld>>2 = %ld\n\t\tMinimum --> %ld>>2 = %ld\n", max_long, max_long>>2, min_long, min_long>>2);
    // & and |
    printf("Complements\n\tIntegers\n\t\tAnd --> %d & %d = %d\n\t\tOr --> %d | %d = %d\n", max_int, min_int, max_int & min_int, max_int, min_int, max_int | min_int);
    printf("\tShorts\n\t\tAnd --> %d & %d = %d\n\t\tOr --> %d | %d = %d\n", max_short, min_short, max_short & min_short, max_short, min_short, max_short | min_short);
    printf("\tLongs\n\t\tAnd --> %ld & %ld = %ld\n\t\tOr --> %ld | %ld = %ld\n", max_long, min_long, max_long & min_long, max_long, min_long, max_long | min_long);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
Complements
        Integers
                Maximum --> ~2147483647 = -2147483648
                Minimum --> ~-2147483648 = 2147483647
        Shorts
                Maximum --> ~32767 = -32768
                Minimum --> ~-32768 = 32767
        Longs
                Maximum --> ~9223372036854775807 = -9223372036854775808
                Minimum --> ~-9223372036854775808 = 9223372036854775807
Left Shift
        Integers
                Maximum --> 2147483647<<2 = -4
                Minimum --> -2147483648<<2 = 0
        Shorts
                Maximum --> 32767<<2 = 131068
                Minimum --> -32768<<2 = -131072
        Longs
                Maximum --> 9223372036854775807<<2 = -4
                Minimum --> -9223372036854775808<<2 = 0
Right Shift
        Integers
                Maximum --> 2147483647>>2 = 536870911
                Minimum --> -2147483648>>2 = -536870912
        Shorts
                Maximum --> 32767>>2 = 8191
                Minimum --> -32768>>2 = -8192
        Longs
                Maximum --> 9223372036854775807>>2 = 2305843009213693951
                Minimum --> -9223372036854775808>>2 = -2305843009213693952
Complements
        Integers
                And --> 2147483647 & -2147483648 = 0
                Or --> 2147483647 | -2147483648 = -1
        Shorts
                And --> 32767 & -32768 = 0
                Or --> 32767 | -32768 = -1
        Longs
                And --> 9223372036854775807 & -9223372036854775808 = 0
                Or --> 9223372036854775807 | -9223372036854775808 = -1
```

e.

i.

```c
#include <stdio.h>
#include <math.h>

int main() {
    float degree = 118.0f;
    float rad;

    rad = degree * (M_PI / 180.0);
    printf("%.3f degrees = %.3f radians\n", degree, rad);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
118.000 degrees = 2.059 radians
```

ii.

```c
#include <stdio.h>
#include <math.h>

int main() {
    float ang_1 = 80.0f;
    float ang_2 = 60.0f;
    float side_1 = 20.0f;
    float side_2 = 30.0f;

    float ang_3 = 180.0f - (ang_1 +  ang_2);
    float side_3 = sqrtf((side_1 * side_1) + (side_2 * side_2) - (2.0 * side_1 * side_2 * cosf(ang_1 * (M_PI / 180.0))));

    printf("Missing side : %.2f\n", side_3);
    printf("Missing angle : %.2f\n", ang_3);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
Missing side : 33.04
Missing angle : 40.00
```

iii.

```c
#include <stdio.h>
#include <math.h>

int main() {
    float side_1 = 20.0f;
    float side_2 = 10.0f;
    float ang_1 = 30.0f * (M_PI / 180.0f);

    float side_3 = sqrtf((side_1 * side_1) + (side_2 * side_2) - (2.0 * side_1 * side_2 * cosf(ang_1)));
    float ang_2 = asinf((side_2 * sin(ang_1)) / side_1) * (180.0f / M_PI);
    ang_1 = ang_1 * (180.0f / M_PI);
    float ang_3 = 180.0f - (ang_1 + ang_2);
    printf("Side 01 : %.2f\nSide 02 : %.2f\nSide 03 : %.2f\n", side_1, side_2, side_3);
    printf("Ang 01 : %.2f\nAng 02 : %.2f\nAng 03 : %.2f\n", ang_1, ang_2, ang_3);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
Side 01 : 20.00
Side 02 : 10.00
Side 03 : 12.39
Ang 01 : 30.00
Ang 02 : 14.48
Ang 03 : 135.52
```

iv.

```c
#include <stdio.h>
#include <math.h>

int main() {
    float num1 = 3.2f;
    float num2 = 14.24f;

    float rem = fmodf(num2, num1);

    printf("%.2f %% %.2f = %.2f\n", num2, num1, rem);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
14.24 % 3.20 = 1.44
```

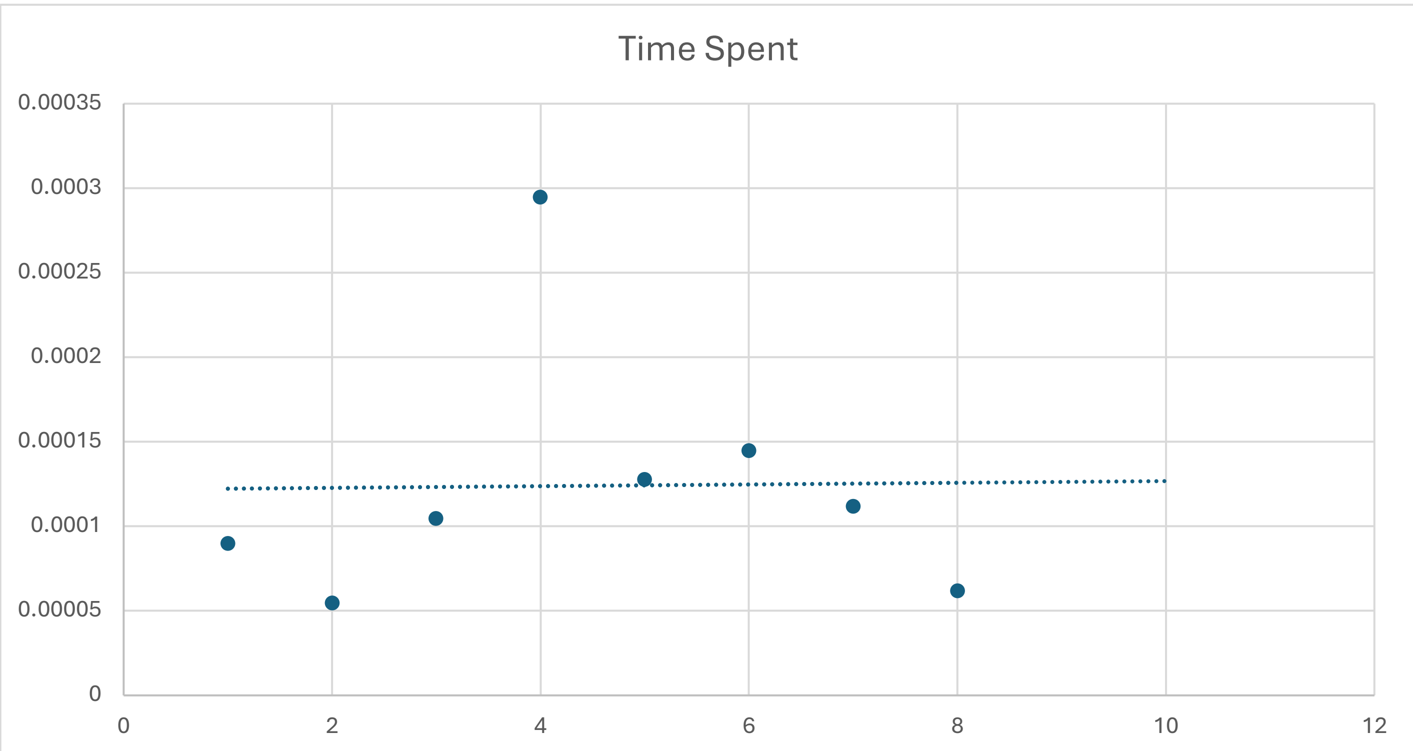
f.

i.

```
Time spent (a) : 0.000090
Time spent (b) : 0.000055
Time spent (c) : 0.000105
Time spent (d) : 0.000295
Time spent (e) (i) : 0.000128
Time spent (e) (ii) : 0.000145
Time spent (e) (iii) : 0.000112
Time spent (e) (iv) : 0.000062
```

ii.

A.



B.

C. clock_t is a fundamental arithmetic type that represents the processor time.

g.

```c
#include <stdio.h>
#include <math.h>

int main() {
    double sal_inc = 500000;
    double add_mon_inc = 200000;

    double taxable_inc = sal_inc - 100000;
    double inc_rate = (taxable_inc / 50000) * 6;
    double salary_rate = fmax(0, fmin(inc_rate, 36));
    double monthly_inc_tax = sal_inc * salary_rate / 100;

    double taxable_add_inc = add_mon_inc - 100000;
    double add_rate = (taxable_add_inc / 50000) * 10;
    double add_inc_rate = fmax(10, fmin(10 + add_rate, 40));
    double monthly_add_tax = add_mon_inc * add_inc_rate / 100;

    printf("Monthly take-home salary : LKR. %.2lf/=\n", sal_inc - monthly_inc_tax);
    printf("Monthly take-home total income : LKR. %.2lf/=\n", (sal_inc - monthly_inc_tax) + (add_mon_inc - monthly_add_tax));
    printf("Total anual income : LKR. %.2lf/=\n", ((sal_inc - monthly_inc_tax) + (add_mon_inc - monthly_add_tax)) * 12);
    printf("Monthly salary tax : LKR. %.2lf/=\n", monthly_inc_tax);
    printf("Anual salary tax : LKR. %.2lf/=\n", monthly_inc_tax * 12);
    printf("Monthly additional income tax : LKR. %.2lf/=\n", monthly_add_tax);
    printf("Anual additional income tax : LKR. %.2lf/=\n", monthly_add_tax * 12);
    printf("Total anual income tax to be paid : LKR. %.2lf/=\n", (monthly_inc_tax +  monthly_add_tax) * 12);
}
```

```
chamal@ubuntuserver:~/Desktop$ ./a.out
Monthly take-home salary : LKR. 320000.00/=
Monthly take-home total income : LKR. 460000.00/=
Total anual income : LKR. 5520000.00/=
Monthly salary tax : LKR. 180000.00/=
Anual salary tax : LKR. 2160000.00/=
Monthly additional income tax : LKR. 60000.00/=
Anual additional income tax : LKR. 720000.00/=
Total anual income tax to be paid : LKR. 2880000.00/=
```

(7)

```c
#include <stdio.h>
//#include <math.h>
```

```
chamal@ubuntuserver:~/Desktop$ gcc ex14.c -lm
ex14.c: In function 'main':
ex14.c:10:30: error: implicit declaration of function 'fmax' [-Wimplicit-function-declaration]
   10 |         double salary_rate = fmax(0, fmin(inc_rate, 36));
      |                              ^~~~
ex14.c:2:1: note: include '<math.h>' or provide a declaration of 'fmax'
    1 | #include <stdio.h>
  +++ |+#include <math.h>
    2 | //#include <math.h>
ex14.c:10:30: warning: incompatible implicit declaration of built-in function 'fmax' [-Wbuiltin-declaration-mismatch]
   10 |         double salary_rate = fmax(0, fmin(inc_rate, 36));
      |                              ^~~~
ex14.c:10:30: note: include '<math.h>' or provide a declaration of 'fmax'
ex14.c:10:38: error: implicit declaration of function 'fmin' [-Wimplicit-function-declaration]
   10 |         double salary_rate = fmax(0, fmin(inc_rate, 36));
      |                                      ^~~~
ex14.c:10:38: note: include '<math.h>' or provide a declaration of 'fmin'
ex14.c:10:38: warning: incompatible implicit declaration of built-in function 'fmin' [-Wbuiltin-declaration-mismatch]
ex14.c:10:38: note: include '<math.h>' or provide a declaration of 'fmin'
```

```
 sal_inc = 500000;
 double add_mon_inc = 200000;
```

```
chamal@ubuntuserver:~/Desktop$ gcc ex14.c -lm
ex14.c: In function ‘main’:
ex14.c:5:9: error: ‘sal_inc’ undeclared (first use in this function)
    5 |         sal_inc = 500000;
      |         ^~~~~~~
ex14.c:5:9: note: each undeclared identifier is reported only once for each function it appears in
```

```
double inc_rate = (taxable_inc // 50000) * 6;
```

```
chamal@ubuntuserver:~/Desktop$ gcc ex14.c -lm
ex14.c: In function ‘main’:
ex14.c:9:39: error: expected ‘)’ before ‘double’
    9 |         double inc_rate = (taxable_inc // 50000) * 6;
      |                           ~           ^
      |                                       )
   10 |         double salary_rate = fmax(0, fmin(inc_rate, 36));
      |         ~~~~~~
ex14.c:26:1: error: expected ‘,’ or ‘;’ before ‘}’ token
   26 | }
      | ^
ex14.c:26:1: error: expected declaration or statement at end of input
```

```
double inc_rate = (taxable_inc / 50000)  6;
```

```
chamal@ubuntuserver:~/Desktop$ gcc ex14.c -lm
ex14.c: In function ‘main’:
ex14.c:9:50: error: expected ‘,’ or ‘;’ before numeric constant
    9 |         double inc_rate = (taxable_inc / 50000)  6;
      |                                                  ^
```

```
double inc_rate = (taxable_inc / 50000) * 6
```

```
chamal@ubuntuserver:~/Desktop$ gcc ex14.c -lm
ex14.c: In function ‘main’:
ex14.c:10:9: error: expected ‘,’ or ‘;’ before ‘double’
   10 |         double salary_rate = fmax(0, fmin(inc_rate, 36));
      |         ^~~~~~
ex14.c:11:44: error: ‘salary_rate’ undeclared (first use in this function)
   11 |         double monthly_inc_tax = sal_inc * salary_rate / 100;
      |                                            ^~~~~~~~~~~
ex14.c:11:44: note: each undeclared identifier is reported only once for each function it appears in
```

```
double salary_rate = fmax(fmin(inc_rate, 36));
```

```
chamal@ubuntuserver:~/Desktop$ gcc ex14.c -lm
ex14.c: In function ‘main’:
ex14.c:10:30: error: too few arguments to function ‘fmax’; expected 2, have 1
   10 |         double salary_rate = fmax(fmin(inc_rate, 36));
      |                              ^~~~
In file included from /usr/include/features.h:539,
                 from /usr/include/aarch64-linux-gnu/bits/libc-header-start.h:33,
                 from /usr/include/stdio.h:28,
                 from ex14.c:1:
/usr/include/aarch64-linux-gnu/bits/mathcalls.h:387:1: note: declared here
  387 | __MATHCALLX (fmax,, (_Mdouble_ __x, _Mdouble_ __y), (__const__));
      | ^~~~~~~~~~~
```

```
printf("Monthly take-home salary : LKR. %.2ld/=\n", sal_inc - monthly_inc_tax);
```

```
chamal@ubuntuserver:~/Desktop$ gcc ex14.c -lm
ex14.c: In function ‘main’:
ex14.c:18:53: warning: format ‘%ld’ expects argument of type ‘long int’, but argument 2 has type ‘double’ [-Wformat=]
   18 |         printf("Monthly take-home salary : LKR. %.2ld/=\n", sal_inc - monthly_inc_tax);
      |                                                 ~~~~^       ~~~~~~~~~~~~~~~~~~~~~~~~~
      |                                                     |               |
      |                                                     long int        double
      |                                                 %.2f
```

```
printf("Monthly take-home salary : LKR. .2lf/=\n", sal_inc - monthly_inc_tax);
```

```
chamal@ubuntuserver:~/Desktop$ gcc ex14.c -lm
ex14.c: In function ‘main’:
ex14.c:18:16: warning: too many arguments for format [-Wformat-extra-args]
   18 |         printf("Monthly take-home salary : LKR. .2lf/=\n", sal_inc - monthly_inc_tax);
      |                ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

```
25     printf("Total anual income tax to be paid : LKR. %.2lf/=\n", (monthly_inc_tax +  monthly_add_tax) * 12);
26 
27
~
```

```
chamal@ubuntuserver:~/Desktop$ gcc ex14.c -lm
ex14.c: In function ‘main’:
ex14.c:25:9: error: expected declaration or statement at end of input
   25 |         printf("Total anual income tax to be paid : LKR. %.2lf/=\n", (monthly_inc_tax +  monthly_add_tax) * 12);
      |         ^~~~~~
```

(8)

a. puts() function is used to print strings on the screen. Unlike printf() function it automatically adds the new line character at the end of the string. Downsides of this are you can only print strings, and no formatting is available in puts() function. to use the function just pass a string as the parameter and the function will print it to the screen.

b.
   - Both prints string to screen
   - Both are in the stdio.h header file

c.
   - Euler's number
   - Pi
   - Golden ratio